

Discussion 1 — Math & Python Foundations

Linear Algebra · Calculus · Probability · Python

EECS 492 · Vaibhav Balloli

LINEAR ALGEBRA

Linear Algebra — The Basics

- **Vectors & matrices:** a matrix $A \in \mathbb{R}^{m \times n}$ has m rows and n columns.
- **Matrix multiplication:** if A is $m \times n$ and B is $n \times p$, then $C = AB$ is $m \times p$ with

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

- Inner dimensions must match; the product is **not** commutative ($AB \neq BA$).

Linear Algebra — Identity, Transpose & Inverse

- **Identity** I : ones on the diagonal, zeros elsewhere; $AI = IA = A$.
- **Transpose** A^T : swap rows and columns; $(AB)^T = B^T A^T$.
- **Inverse** A^{-1} : satisfies $AA^{-1} = A^{-1}A = I$. Only *square* matrices can have one, and many don't.

Linear Algebra — Problem

Let

$$A = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}_{3 \times 4}, \quad B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}_{4 \times 3}.$$

Compute the product $C = AB$ (a 3×3 matrix).

Linear Algebra — Solution: the product

Using $C_{ij} = \sum_k A_{ik} B_{kj}$ (row i of A dotted with column j of B):

$$C = AB = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

Check the shapes: A is 3×4 and B is 4×3 , so the inner dimensions (4) match and the product is 3×3 . ✓

CALCULUS

Calculus — Derivatives

Single & multivariable

- **Derivative:** $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$; rules — power, product, and **chain:** $\frac{d}{dx} g(h(x)) = g'(h) h'(x)$.
- **Partial derivative** $\partial f / \partial x$: differentiate w.r.t. one variable, hold others fixed.
- **Gradient:** $\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$ — direction of steepest ascent; gradient descent steps toward a minimum.

Calculus — Common Derivatives

Primers worth memorizing (all follow from the limit definition + chain rule):

- **Power:** $\frac{d}{dx} x^n = n x^{n-1}$
- **Exponential / log:** $\frac{d}{dx} e^x = e^x$, $\frac{d}{dx} a^x = a^x \ln a$, $\frac{d}{dx} \ln x = \frac{1}{x}$
- **Trig:** $\frac{d}{dx} \sin x = \cos x$, $\frac{d}{dx} \cos x = -\sin x$

Calculus — The Gradient

The calculus behind optimization:

- **Gradient** of a scalar $f: \mathbb{R}^n \rightarrow \mathbb{R}$: $\nabla_{\mathbf{x}} f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$
— collect every partial derivative into one vector.

Calculus — Multivariate Chain Rule

When $z = f(x, y)$ and both $x = x(t)$ and $y = y(t)$ depend on t , the rate of change of z **adds up the contribution through each variable**:

$$\frac{dz}{dt} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}.$$

Example. $f(x, y) = x^2y$ with $x = t$ and $y = t^2$, so $\frac{dx}{dt} = 1$ and $\frac{dy}{dt} = 2t$:
$$\frac{dz}{dt} = \underbrace{(2xy)}_{\partial f / \partial x} (1) + \underbrace{(x^2)}_{\partial f / \partial y} (2t) = 2(t)(t^2) + (t^2)(2t) = 4t^3.$$

Check by substituting first: $z = x^2y = t^2 \cdot t^2 = t^4$, so $\frac{dz}{dt} = 4t^3$. ✓

Calculus — Problem

Let

$$f(x, y) = x^2y + 3xy^3.$$

1. (Partials & gradient) Find $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$, then evaluate ∇f at $(1, 2)$.

2. (Chain rule with a logarithm) Let $\sigma(z) = \frac{1}{1+e^{-z}}$.

(a) Show that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

(b) Compute $\frac{d}{dz} [-\ln \sigma(z)]$.

Calculus — Solution (1): partials & gradient

$$\frac{\partial f}{\partial x} = 2xy + 3y^3, \quad \frac{\partial f}{\partial y} = x^2 + 9xy^2.$$

At (1,2):

$$\frac{\partial f}{\partial x} = 2(1)(2) + 3(8) = 28, \quad \frac{\partial f}{\partial y} = 1 + 9(1)(4) = 37.$$

$$\nabla f(1,2) = (28, 37).$$

Calculus — Solution (2): chain rule with a logarithm

(a) Write $\sigma(z) = (1 + e^{-z})^{-1}$ and apply the chain rule:

$$\begin{aligned}\sigma'(z) &= -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \sigma(z)(1 - \sigma(z)).\end{aligned}$$

(b) Using $\frac{d}{du} \ln u = \frac{1}{u}$ and part (a):

$$\frac{d}{dz} [-\ln \sigma(z)] = -\frac{\sigma'(z)}{\sigma(z)} = -(1 - \sigma(z)) = \sigma(z) - 1.$$

Calculus — Multivariate Practice

For each scalar function of several variables, find **every first partial derivative** and assemble the **gradient**:

1. $g(x, y) = e^{xy} + x^2y$

2. $h(x, y) = \ln(x^2 + y^2)$

3. $L(w, b) = (wx + b - y)^2$ — here x, y are fixed constants (a first taste of fitting a line to data)

Calculus — Multivariate Practice: Solution (1)

1. $g(x, y) = e^{xy} + x^2y$ — treat the other variable as a constant; the e^{xy} term needs the chain rule.

$$\frac{\partial g}{\partial x} = y e^{xy} + 2xy, \quad \frac{\partial g}{\partial y} = x e^{xy} + x^2,$$

$$\nabla g = (y e^{xy} + 2xy, \quad x e^{xy} + x^2).$$

2. $h(x, y) = \ln(x^2 + y^2)$ — chain rule with $d/du \ln u = 1/u$:

$$\frac{\partial h}{\partial x} = \frac{2x}{x^2 + y^2}, \quad \frac{\partial h}{\partial y} = \frac{2y}{x^2 + y^2} \Rightarrow \nabla h = \frac{2}{x^2 + y^2} (x, y).$$

Calculus — Multivariate Practice: Solution (2)

3. $L(w, b) = (wx + b - y)^2$. Let $r = wx + b - y$ (the **residual**) and apply the chain rule, remembering x, y are constants:

$$\frac{\partial L}{\partial w} = 2r \cdot \frac{\partial r}{\partial w} = 2(wx + b - y) x,$$
$$\frac{\partial L}{\partial b} = 2r \cdot \frac{\partial r}{\partial b} = 2(wx + b - y).$$

So $\nabla L = (2rx, 2r)$.

PROBABILITY

Probability — The Basics

- **Conditional probability:** $P(A \mid B) = \frac{P(A \cap B)}{P(B)}$, for $P(B) > 0$.
- **Independence:** $A \perp B \Leftrightarrow P(A \cap B) = P(A)P(B)$.
- **Law of total probability:** for a partition $\{B_i\}$,
- $P(A) = \sum_i P(A \mid B_i) P(B_i)$.

Probability — Bayes' Theorem

Update a belief given evidence:

$$P(H \mid E) = \frac{P(E \mid H) P(H)}{P(E)} = \frac{P(E \mid H) P(H)}{\sum_i P(E \mid H_i) P(H_i)}.$$

Read it as: **posterior** $\propto$ **likelihood** $\times$ **prior**, rescaled so probabilities sum to 1.

Probability — Vocabulary

H = **hypothesis** (what we want to know, e.g. “has the disease”);

E = **evidence** (what we observe, e.g. “tested positive”).

- **Prior** $P(H)$ — belief *before* evidence (the base rate).
- **Likelihood** $P(E \mid H)$ — how well H explains E .
- **Evidence** $P(E)$ — normalizer, $\sum_i P(E \mid H_i)P(H_i)$.
- **Posterior** $P(H \mid E)$ — updated belief *after* evidence.

Probability — Problem

A disease affects **1%** of a population. A test has:

- **Sensitivity** $P(+ | D) = 0.99$ (true positive rate)
- **False-positive rate** $P(+ | \neg D) = 0.05$

A person tests positive. **What is** $P(D | +)$, the probability they actually have the disease?

Probability — Solution (1): apply Bayes

Goal: the posterior $P(D \mid +)$. Bayes needs the overall chance of testing positive, $P(+)$.

Step 1 — base rate (prior): $P(D) = 0.01$, so $P(\neg D) = 0.99$.

Step 2 — total probability of a positive test (true positives + false positives):

$$\begin{aligned} P(+) &= P(+ \mid D)P(D) + P(+ \mid \neg D)P(\neg D) = (0.99)(0.01) + (0.05)(0.99) \\ &\quad \text{true} \qquad \qquad \qquad \text{false} \\ &= 0.0099 + 0.0495 = 0.0594. \end{aligned}$$

Step 3 — posterior via Bayes:

$$P(D \mid +) = \frac{P(+ \mid D) P(D)}{P(+)} = \frac{0.0099}{0.0594} \approx 0.167.$$

Probability — Solution (2): why only ~17%?

Same numbers, counted as people. Imagine **10,000** tested:

- **100** actually have the disease (1%); of these, 99% test positive → **99 true positives**.
- **9,900** are healthy; 5% still test positive → **495 false positives**.
- Positives total $99 + 495 = 594$, but only 99 are truly sick.

$$P(D \mid +) = \frac{99}{594} \approx 0.167.$$

The false positives come from a *much* larger healthy group, so they swamp the true positives. When a condition is **rare**, even a very accurate test leaves you likely healthy after one positive — the **base rate** dominates.

PYTHON

Python — NumPy Operations

NumPy is the core library for numerical computing — fast, vectorized `ndarrays`.

- **Creation:** `np.array(...)`, `np.zeros`, `np.ones`, `np.arange`, `np.eye`
- **Shape & indexing:** `A.shape`, `A[i, j]`, slicing `A[:, 1]`, `A.reshape(m, n)`
- **Broadcasting / elementwise:** `v * 2`, `A + 1`, `A * B` (the **Hadamard product** $A \odot B$ — element-wise, *not* matrix multiplication)
- **Linear algebra:** `A @ B` (or `np.dot`) for matrix products, `A.T` for the transpose
- **Aggregations:** `A.sum()`, `A.mean(axis=0)`, `A.max()`

Python — NumPy in Action

Reproduce the Linear Algebra problem in code: build A (3×4) and B (4×3), compute $C = A @ B$, check `C.shape == (3, 3)`, and confirm the result matches the hand computation with `np.allclose`.